

Python for Data Science - 2305CS303

Lab - 11

Roll No. : 128

Name : Riyaz

1. Create a MultIndexed DataFrame

Create a DataFrame containing student marks for different Departments and Semesters. Use `set_index()` to create a MultiIndex using `Department` and `Semester`. Display the DataFrame and its index.

```
In [1]: import pandas as pd

import numpy as np

data = {
    "RollNo": [101, 102, 103, 104, 105, 106],
    "Name": ["Aarav", "Bhavya", "Chetan", "Dev", "Esha", "Farhan"],
    "Department": ["CSE", "ECE", None, "ME", "ECE", "CSE"],
    "Year": [1, 2, 1, 2, 1, 2],
    "Gender": ["Male", "Female", "Female", "Male", "Male", "Female"],
    "Semester": [3, 4, 3, 4, 3, 4],
    "Marks": [85, np.nan, 65, 88, 75, 58],
    "City": ["Mumbai", "Delhi", "Bangalore", "Chennai", None, "Pune"],
    "Subject": ["Python", "DBMS", "Network", "Java", "Maths", "OS"]
}

students = pd.DataFrame(data)

students.set_index(["Department", "Semester"], inplace=True)

print(students)
print(students.index)
```

Department	Semester	RollNo	Name	Year	Gender	Marks	City	Subject
CSE	3	101	Aarav	1	Male	85.0	Mumbai	Python
ECE	4	102	Bhavya	2	Female	NaN	Delhi	DBMS
NaN	3	103	Chetan	1	Female	65.0	Bangalore	Network
ME	4	104	Dev	2	Male	88.0	Chennai	Java
ECE	3	105	Esha	1	Male	75.0	None	Maths
CSE	4	106	Farhan	2	Female	58.0	Pune	OS

```
MultiIndex([('CSE', 3),
            ('ECE', 4),
            (nan, 3),
            ('ME', 4),
            ('ECE', 3),
            ('CSE', 4)],
            names=['Department', 'Semester'])
```

2. Create a MultiIndex using Multiple Columns

Create a DataFrame containing `Department` , `Year` , `Gender` , `Student` and `Marks` . Set `Department` , `Year` and `Gender` as the MultiIndex and display the resulting DataFrame.

```
In [2]: students = pd.DataFrame(data)

students.set_index(["Department", "Year", "Gender"], inplace=True)

print(students)
```

Department	Year	Gender	RollNo	Name	Semester	Marks	City	Subject
CSE	1	Male	101	Aarav	3	85.0	Mumbai	Python
ECE	2	Female	102	Bhavya	4	NaN	Delhi	DBMS
NaN	1	Female	103	Chetan	3	65.0	Bangalore	Network
ME	2	Male	104	Dev	4	88.0	Chennai	Java
ECE	1	Male	105	Esha	3	75.0	None	Maths
CSE	2	Female	106	Farhan	4	58.0	Pune	OS

3. Read a MultiIndexed DataFrame Directly from CSV

Create a CSV file containing columns `Department` , `Semester` , `Subject` , and `Marks` . Read the CSV file directly as a MultiIndexed DataFrame using `pd.read_csv()` with the appropriate `index_col` parameter. Display the resulting DataFrame.

```
In [3]: df = pd.DataFrame(data)

df.to_csv("marks.csv", index=False)
students = pd.read_csv(
    "marks.csv",
    index_col=["Department", "Semester", "Subject"]
)

print(students)
```

			RollNo	Name	Year	Gender	Marks	City
Department	Semester	Subject						
CSE	3	Python	101	Aarav	1	Male	85.0	Mumbai
ECE	4	DBMS	102	Bhavya	2	Female	NaN	Delhi
NaN	3	Network	103	Chetan	1	Female	65.0	Bangalore
ME	4	Java	104	Dev	2	Male	88.0	Chennai
ECE	3	Maths	105	Esha	1	Male	75.0	NaN
CSE	4	OS	106	Farhan	2	Female	58.0	Pune

4. Inspect a MultiIndexed DataFrame

Using the MultiIndexed DataFrame created in Question 3, display the index levels, index names, columns, shape, and data types.

```
In [4]: # Display index levels
print(students.index.levels)

# Display index names
print(students.index.names)

# Display columns
print(students.columns)

# Display shape
print(students.shape)

# Display data types
print(students.dtypes)
```

```
[['CSE', 'ECE', 'ME'], [3, 4], ['DBMS', 'Java', 'Maths', 'Network', 'OS', 'Python']]
['Department', 'Semester', 'Subject']
Index(['RollNo', 'Name', 'Year', 'Gender', 'Marks', 'City'], dtype='object')
(6, 6)
RollNo      int64
Name        object
Year        int64
Gender      object
Marks       float64
City        object
dtype: object
```

5. Access Data using MultiIndex

Using a MultiIndexed DataFrame having `Department` and `Semester` as indexes, access all records belonging to one department and one specific department-semester combination.

```
In [5]: students = pd.DataFrame(data)

students.set_index(["Department", "Semester"], inplace=True)

print(students)
print(students.loc[('CSE', 3)])
```

		RollNo	Name	Year	Gender	Marks	City	Subject
Department	Semester							
CSE	3	101	Aarav	1	Male	85.0	Mumbai	Python
ECE	4	102	Bhavya	2	Female	NaN	Delhi	DBMS
NaN	3	103	Chetan	1	Female	65.0	Bangalore	Network
ME	4	104	Dev	2	Male	88.0	Chennai	Java
ECE	3	105	Esha	1	Male	75.0	None	Maths
CSE	4	106	Farhan	2	Female	58.0	Pune	OS

RollNo 101
 Name Aarav
 Year 1
 Gender Male
 Marks 85.0
 City Mumbai
 Subject Python
 Name: (CSE, 3), dtype: object

6. Cross Section using xs()

Create a MultiIndexed DataFrame with `Department` and `Semester` as index levels. Use `xs()` to retrieve the records for a particular semester across all departments.

```
In [6]: students = pd.DataFrame(data)

students.set_index(["Department", "Semester"], inplace=True)

print(students.xs(4, level="Semester"))
```

	RollNo	Name	Year	Gender	Marks	City	Subject
Department							
ECE	102	Bhavya	2	Female	NaN	Delhi	DBMS
ME	104	Dev	2	Male	88.0	Chennai	Java
CSE	106	Farhan	2	Female	58.0	Pune	OS

7. Cross Section from a Specific Level

Using the same MultiIndexed DataFrame, use `xs()` with the `level` parameter to retrieve records for one department. Display the result.

```
In [7]: students = pd.DataFrame(data)

students.set_index(["Department", "Semester"], inplace=True)

print(students.xs("CSE", level="Department"))
```

	RollNo	Name	Year	Gender	Marks	City	Subject
Semester							
3	101	Aarav	1	Male	85.0	Mumbai	Python
4	106	Farhan	2	Female	58.0	Pune	OS

8. Cross Section on Multiple Index Levels

Create a MultiIndexed DataFrame with `Department`, `Semester`, and `Subject` as index levels. Use `xs()` to retrieve the records corresponding to a particular combination of Department and Semester.

10. Count Missing Values

Using the DataFrame from Question 9, calculate the number of missing values in each column and display the total number of missing values in the complete DataFrame.

```
In [10]: print(students.isna().sum())

print("Total Missing Values:", students.isna().sum().sum())
```

RollNo	0
Name	0
Department	1
Year	0
Gender	0
Semester	0
Marks	1
City	1
Subject	0

dtype: int64
Total Missing Values: 3

11. Detect Missing Values using isnull()

Create a DataFrame containing missing values in different columns. Use `isnull()` and `notnull()` to display the missing and non-missing values. Compare the results of `isnull()` and `isna()`.

```
In [11]: students = pd.DataFrame(data)

print("Missing Values:")
print(students.isnull())

print("\nNon-Missing Values:")
print(students.notnull())

print("\nUsing isna():")
print(students.isna())

print("\nComparison:")
print(students.isnull().equals(students.isna()))
```

Missing Values:

	RollNo	Name	Department	Year	Gender	Semester	Marks	City	Subject
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	True	False	False
2	False	False	True	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	True	False
5	False	False	False	False	False	False	False	False	False

Non-Missing Values:

	RollNo	Name	Department	Year	Gender	Semester	Marks	City	Subject
0	True	True	True	True	True	True	True	True	True
1	True	True	True	True	True	True	False	True	True
2	True	True	False	True	True	True	True	True	True
3	True	True	True	True	True	True	True	True	True
4	True	True	True	True	True	True	True	False	True
5	True	True	True	True	True	True	True	True	True

Using isna():

	RollNo	Name	Department	Year	Gender	Semester	Marks	City	Subject
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	True	False	False
2	False	False	True	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	True	False
5	False	False	False	False	False	False	False	False	False

Comparison:

True

12. Remove Missing Values

Using a DataFrame containing missing values, demonstrate `dropna()` to remove rows containing at least one missing value. Also demonstrate how to remove columns containing missing values.

```
In [12]: students = pd.DataFrame(data)

print("Original DataFrame:")
print(students)

rows_removed = students.dropna()

print("\nAfter Removing Rows:")
print(rows_removed)

columns_removed = students.dropna(axis=1)

print(columns_removed)
```

Original DataFrame:

	RollNo	Name	Department	Year	Gender	Semester	Marks	City	\
0	101	Aarav	CSE	1	Male	3	85.0	Mumbai	
1	102	Bhavya	ECE	2	Female	4	NaN	Delhi	
2	103	Chetan	None	1	Female	3	65.0	Bangalore	
3	104	Dev	ME	2	Male	4	88.0	Chennai	
4	105	Esha	ECE	1	Male	3	75.0	None	
5	106	Farhan	CSE	2	Female	4	58.0	Pune	

	Subject
0	Python
1	DBMS
2	Network
3	Java
4	Maths
5	OS

After Removing Rows:

	RollNo	Name	Department	Year	Gender	Semester	Marks	City	Subject
0	101	Aarav	CSE	1	Male	3	85.0	Mumbai	Python
3	104	Dev	ME	2	Male	4	88.0	Chennai	Java
5	106	Farhan	CSE	2	Female	4	58.0	Pune	OS

	RollNo	Name	Year	Gender	Semester	Subject
0	101	Aarav	1	Male	3	Python
1	102	Bhavya	2	Female	4	DBMS
2	103	Chetan	1	Female	3	Network
3	104	Dev	2	Male	4	Java
4	105	Esha	1	Male	3	Maths
5	106	Farhan	2	Female	4	OS

13. Fill Missing Values with a Constant

Create a DataFrame containing missing values in the `Marks` and `City` columns.

Replace missing values using suitable constant values with `fillna()` .

```
In [13]: students = pd.DataFrame(data)

print("Before Filling:")
print(students)

students["Marks"].fillna(0)
students["City"].fillna("Unknown")

print("\nAfter Filling:")
print(students)
```

Before Filling:

	RollNo	Name	Department	Year	Gender	Semester	Marks	City	\
0	101	Aarav	CSE	1	Male	3	85.0	Mumbai	
1	102	Bhavya	ECE	2	Female	4	NaN	Delhi	
2	103	Chetan	None	1	Female	3	65.0	Bangalore	
3	104	Dev	ME	2	Male	4	88.0	Chennai	
4	105	Esha	ECE	1	Male	3	75.0	None	
5	106	Farhan	CSE	2	Female	4	58.0	Pune	

Subject

0	Python
1	DBMS
2	Network
3	Java
4	Maths
5	OS

After Filling:

	RollNo	Name	Department	Year	Gender	Semester	Marks	City	\
0	101	Aarav	CSE	1	Male	3	85.0	Mumbai	
1	102	Bhavya	ECE	2	Female	4	NaN	Delhi	
2	103	Chetan	None	1	Female	3	65.0	Bangalore	
3	104	Dev	ME	2	Male	4	88.0	Chennai	
4	105	Esha	ECE	1	Male	3	75.0	None	
5	106	Farhan	CSE	2	Female	4	58.0	Pune	

Subject

0	Python
1	DBMS
2	Network
3	Java
4	Maths
5	OS

14. Fill Missing Values using Mean and Median

Create a DataFrame containing missing numerical values. Fill missing values in the **Marks** column using the mean and demonstrate filling another numerical column using the median.

```
In [14]: students = pd.DataFrame(data)

# Fill Marks using Mean
mean_marks = students["Marks"].mean()
students["Marks"] = students["Marks"].fillna(mean_marks)

# Fill Semester using Median
median_semester = students["Semester"].median()
students["Semester"] = students["Semester"].fillna(median_semester)

print(students)
```

	RollNo	Name	Department	Year	Gender	Semester	Marks	City \
0	101	Aarav	CSE	1	Male	3	85.0	Mumbai
1	102	Bhavya	ECE	2	Female	4	74.2	Delhi
2	103	Chetan	None	1	Female	3	65.0	Bangalore
3	104	Dev	ME	2	Male	4	88.0	Chennai
4	105	Esha	ECE	1	Male	3	75.0	None
5	106	Farhan	CSE	2	Female	4	58.0	Pune

	Subject
0	Python
1	DBMS
2	Network
3	Java
4	Maths
5	OS

15. Fill Missing Categorical Values using Mode

Create a DataFrame containing missing values in a categorical column such as `City` or `Department` . Fill the missing values using the mode of that column.

```
In [15]: students = pd.DataFrame(data)

city_mode = students["City"].mode()[0]
students["City"] = students["City"].fillna(city_mode)
print(students)
```

	RollNo	Name	Department	Year	Gender	Semester	Marks	City \
0	101	Aarav	CSE	1	Male	3	85.0	Mumbai
1	102	Bhavya	ECE	2	Female	4	NaN	Delhi
2	103	Chetan	None	1	Female	3	65.0	Bangalore
3	104	Dev	ME	2	Male	4	88.0	Chennai
4	105	Esha	ECE	1	Male	3	75.0	Bangalore
5	106	Farhan	CSE	2	Female	4	58.0	Pune

	Subject
0	Python
1	DBMS
2	Network
3	Java
4	Maths
5	OS

16. Forward Fill and Backward Fill

Create a time-series-like DataFrame containing missing values. Demonstrate forward filling using `ffill()` and backward filling using `bfill()` . Observe the difference between the two methods.

```
In [16]: data_time_series = {
    "Date": ["2026-09-01", "2026-09-02", "2026-09-03",
            "2026-09-04", "2026-09-05"],
    "Marks": [80, np.nan, 90, np.nan, 85]
}

d = pd.DataFrame(data_time_series)
print(d)
```

```

forward = d.ffill()

print("\nAfter Forward Fill:")
print(forward)

backward = d.bfill()
print("\nAfter Backward Fill:")
print(backward)

```

	Date	Marks
0	2026-09-01	80.0
1	2026-09-02	NaN
2	2026-09-03	90.0
3	2026-09-04	NaN
4	2026-09-05	85.0

After Forward Fill:

	Date	Marks
0	2026-09-01	80.0
1	2026-09-02	80.0
2	2026-09-03	90.0
3	2026-09-04	90.0
4	2026-09-05	85.0

After Backward Fill:

	Date	Marks
0	2026-09-01	80.0
1	2026-09-02	90.0
2	2026-09-03	90.0
3	2026-09-04	85.0
4	2026-09-05	85.0

17. Groupby - Basic Grouping

Create a DataFrame containing `Department`, `Student`, and `Marks`. Group the data by `Department` and calculate the average marks for each department.

```

In [20]: students = pd.DataFrame(data)
result = students.groupby("Department")["Marks"].mean()

print(result)

```

```

Department
CSE      71.5
ECE      75.0
ME       88.0
Name: Marks, dtype: float64

```

18. Groupby with Multiple Aggregations

Using a student DataFrame, group the records by `Department` and calculate the minimum, maximum, mean, and count of `Marks` for each department.

```

In [18]: students = pd.DataFrame(data)

result = students.groupby("Department")["Marks"].agg( ["min", "max", "mean", "co

```

```
print(result)
```

	min	max	mean	count
Department				
CSE	58.0	85.0	71.5	2
ECE	75.0	75.0	75.0	1
ME	88.0	88.0	88.0	1

19. Groupby with Multiple Columns

Create a DataFrame containing `Department`, `Semester`, `Student`, and `Marks`.

Group the data by both `Department` and `Semester` and calculate the average marks.

```
In [19]: students = pd.DataFrame(data)

result = students.groupby(
    ["Department", "Semester"]
)["Marks"].mean()

print(result)
```

Department	Semester	
CSE	3	85.0
	4	58.0
ECE	3	75.0
	4	NaN
ME	4	88.0

Name: Marks, dtype: float64

20. Named Aggregation using agg()

Using a student DataFrame, group the data by `Department` and use `agg()` with named aggregation to calculate `Maximum_Marks`, `Minimum_Marks`, and `Average_Marks`.

```
In [21]: result = students.groupby("Department").agg(
    Maximum_Marks=("Marks", "max"),
    Minimum_Marks=("Marks", "min"),
    Average_Marks=("Marks", "mean")
)

print(result)
```

	Maximum_Marks	Minimum_Marks	Average_Marks
Department			
CSE	85.0	58.0	71.5
ECE	75.0	75.0	75.0
ME	88.0	88.0	88.0

21. Groupby and Count Records

Create a DataFrame containing student details and departments. Use `groupby()` to count the number of students in each department. Also calculate the number of unique cities for each department.

```
In [23]: result = students.groupby("Department").agg(
    Student_Count=("Name", "count"),
    Unique_Cities=("City", "nunique")
)

print(result)
```

	Student_Count	Unique_Cities
Department		
CSE	2	2
ECE	2	1
ME	1	1

22. Groupby after Handling Missing Data

Create a DataFrame containing `Department`, `Marks`, and `City` with missing values. Handle the missing marks appropriately and then group the data by department to calculate average marks.

```
In [30]: mean_marks = students["Marks"].mean()
students["Marks"] = students["Marks"].fillna(mean_marks)

result = students.groupby("Department")["Marks"].mean()

print(result)
```

Department	
CSE	71.5
ECE	74.6
ME	88.0

Name: Marks, dtype: float64

In []: